

Empirical Evaluation of PyNear:

Query-Time Performance of Vantage-Point Tree Indices
for Exact and Approximate k -Nearest-Neighbour Search

PyNear Development Team

<https://github.com/pablocael/pynear>

Third edition — PyNear version 2.5

July 2026

Abstract

We present a systematic empirical evaluation of *PyNear*, a Python library implementing exact and approximate k -nearest-neighbour (k -NN) search via Vantage-Point Trees (VP-Trees), IVF-style approximate indices, and Multi-Index Hashing, with SIMD-accelerated distance kernels. This third edition of the report re-measures every experiment with PyNear version 2.5, whose exact indices gained *leaf bucketing*: tree splitting stops at 32-point leaves, which are then scanned as contiguous SIMD sweeps, combined with best-first descent and pooled per-query heaps. PyNear is benchmarked against three widely adopted libraries — *Faiss*, *scikit-learn*, and *Annoy* — across five experimental configurations spanning the Euclidean (ℓ_2), Manhattan (ℓ_1), and Hamming distance metrics, with dataset cardinalities up to $N = 2.5 \times 10^6$ and dimensionalities from $d = 2$ to $d = 1024$. In the exact-search regime, `VPtreeL2Index` answers a 16-query batch over $N = 2.5 \times 10^6$ points in 0.4–0.9 ms across $d = 2$ –16 — 9–20 $\times$ faster than Faiss’s BLAS-backed `IndexFlatL2` scan measured in the same process — and at $d = 128$ ($N = 120\,000$) it is 11 $\times$ faster than Faiss and 83 $\times$ faster than *scikit-learn*. For exact Hamming search, Faiss `IndexBinaryFlat` remains the fastest method at every tested width when measured fairly in its own process (0.15–0.29 ms vs. 7.6–15.9 ms for `VPtreeBinaryIndex`); the apparent 256-bit win for the VP-tree in the in-process suite is an artefact of the libgomp/libomp contention discussed in Section 2. In the high-dimensionality approximate regime ($d = 128$ –1024), `IVFFlatL2Index` attains perfect Recall@10 at $n_{\text{probe}} = 20$ with 5.6–21.5 ms for 32 queries, but Faiss `IndexIVFFlat` is now 8–32 $\times$ faster on raw latency at every tested dimensionality; PyNear’s IVF builds, however, remain 1.4–2.4 $\times$ faster than Faiss at $N = 50\,000$. For approximate binary-descriptor search, `MIHBinaryIndex` retrieves 512-bit near-duplicates at $N = 10^6$ in 0.0088 ms/query (114 039 QPS) with 100% Recall@10 — 34 $\times$ faster than a thread-matched, subprocess-isolated Faiss `IndexBinaryFlat` brute-force scan — while on narrow 128-bit SIFT1M descriptors at high recall an optimised brute-force scan remains the better choice. All experiments were conducted on an Intel[®] Core[™] Ultra 9 285K (24 cores).

Contents

1	Introduction	3
2	Experimental Setup	3
3	Exact-Search Benchmarks	4
3.1	Euclidean Distance — Low Dimensionality	4

3.2	Euclidean Distance — High Dimensionality	5
3.3	Manhattan Distance (ℓ_1)	5
3.4	Hamming Distance — Binary Descriptors	6
3.5	Internal Distance-Metric Comparison	7
4	Approximate-Search Benchmarks	7
4.1	Query Latency vs. Dimensionality at Fixed Probe Budget	8
4.2	Summary: Recall and Latency at $n_{\text{probe}} = 20$	8
4.3	Recall-Latency Pareto Analysis	9
4.4	Index Build Time at Fixed Dataset Size	10
5	Approximate Binary-Descriptor Search	10
5.1	Wide Descriptors: 512-bit Near-Duplicate Retrieval ($N = 10^6$)	11
5.2	Narrow Descriptors: SIFT1M, 128-bit Sign-Quantised ($N = 10^6$)	12
5.3	MIH Parametrisation Guidelines	12
6	Build-Time Scalability	13
6.1	Exact Index: VPTreeL2Index vs. Faiss IndexFlatL2	13
6.2	Approximate Index: IVFFlatL2Index vs. Faiss IndexIVFFlat	14
7	Discussion	15
8	Conclusion	16

1 Introduction

Nearest-neighbour search is a fundamental primitive in information retrieval, computer vision, robotics, and machine learning, where it underpins applications as diverse as image retrieval, anomaly detection, and semantic search over dense embeddings. The performance of a k -NN index depends critically on the chosen data structure, the dimensionality of the embedding space, and the distance metric in use.

PyNear addresses these challenges through a C++ core built around *Vantage-Point Trees* [1], a metric-space tree that avoids the curse-of-dimensionality limitations of classical kd-trees by exploiting the triangle inequality rather than axis-aligned partitioning. SIMD intrinsics (AVX2 on x86-64) further accelerate the hot distance computation paths. The library exposes a uniform Python API for both exact `VPtree*` indices and the approximate `IVFFlat*` family, which implements an inverted file (IVF) scheme over Voronoi cells with a configurable probe budget, alongside `MIHBinaryIndex` (Multi-Index Hashing) for approximate Hamming-distance search.

Version 2.5, evaluated in this edition, reworks the exact-index hot paths: tree splitting now stops at 32-point *leaf buckets* that are scanned as contiguous SIMD sweeps (the strategy used by Faiss and scikit-learn leaf nodes), combined with direct mandatory-child descent, pooled per-query heaps, and release of the Python GIL around all heavy C++ work. Earlier editions introduced the BLAS-backed inner-cluster flat scan of `IVFFlatL2Index` (NumPy SGEMV identity $\|q - x\|^2 = \|q\|^2 + \|x\|^2 - 2q^\top x$), a native C++ K-Means++ clustering kernel, and the approximate binary indices (`MIHBinaryIndex`, `IVFFlatBinaryIndex`); all of these are re-measured here on the same machine as the previous edition.

The remainder of this report is organised as follows. Section 2 describes the experimental setup. Section 3 reports results for exact-search indices across ℓ_2 , ℓ_1 , and Hamming metrics. Section 4 evaluates the approximate ℓ_2 regime. Section 5 presents the approximate binary-descriptor indices (`MIHBinaryIndex`, `IVFFlatBinaryIndex`). Section 6 analyses build-time scalability from $N = 10\,000$ to $N = 10^6$. Section 7 synthesises the findings, and Section 8 concludes.

2 Experimental Setup

Hardware. All experiments were executed on a workstation equipped with an Intel[®] Core[™] Ultra 9 285K processor (24 cores). The Arrow Lake microarchitecture does not support AVX-512; PyNear was built with its deterministic AVX2 baseline flags (`-mavx2 -mfma -mpopcnt`), so all SIMD kernels use AVX2/FMA and hardware popcount. No GPU acceleration was used; all libraries ran on the CPU.

Competitors. We compare against the following external libraries:

- **Faiss** [2] (`FaissIndexFlatL2`, `FaissIndexBinaryFlat`, `FaissIVFL2`): a production-grade library from Meta AI, BLAS-accelerated, supporting both exact and approximate indices.
- **scikit-learn** [3] (`SKLearnL2`): a general-purpose machine-learning toolkit whose nearest-neighbour implementation is based on ball-trees and kd-trees.
- **Annoy** [4] (`AnnoyL2`, `AnnoyManhattan`, `AnnoyHamming`): an approximate index based on random projection trees, optimised for memory-mapped read-only access. Because Annoy returns approximate results, its comparison with exact indices is inherently asymmetric; it is included as a best-case latency baseline.

Protocol. Query time is reported as the wall-clock time (in milliseconds) to answer a batch of n_q queries with k nearest neighbours, averaged over eight repetitions with outlier rejection. Dataset points are drawn from a synthetic clustered distribution so that the hardness of the problem is representative of real-world scenarios. The exact parameters for each benchmark are reported in the caption of the corresponding figure.

OpenMP runtime caveat. PyNear links GNU `libgomp` while `faiss-cpu` bundles LLVM `libomp`. When both are loaded into a single Python process, the two OpenMP runtimes contend and can serialise Faiss’s parallel flat scans. The exact-search suites of Section 3 run all libraries in one process (the historical protocol), so the Faiss flat-scan numbers reported there should be read as *in-process* figures that may be depressed by this contention. The approximate binary-descriptor comparison of Section 5 therefore times Faiss `IndexBinaryFlat` in a separate Faiss-only subprocess, which is the fair configuration for that baseline.

3 Exact-Search Benchmarks

3.1 Euclidean Distance — Low Dimensionality

Figure 1 shows query latency as a function of dimensionality for $d \in \{2, 3, 4, 5, 6, 7, 8, 16\}$ with $N = 2.5 \times 10^6$ data points, $k = 8$, and $n_q = 16$ queries. Note that this edition enlarges the dataset five-fold with respect to the previous one ($N = 5 \times 10^5 \rightarrow 2.5 \times 10^6$).

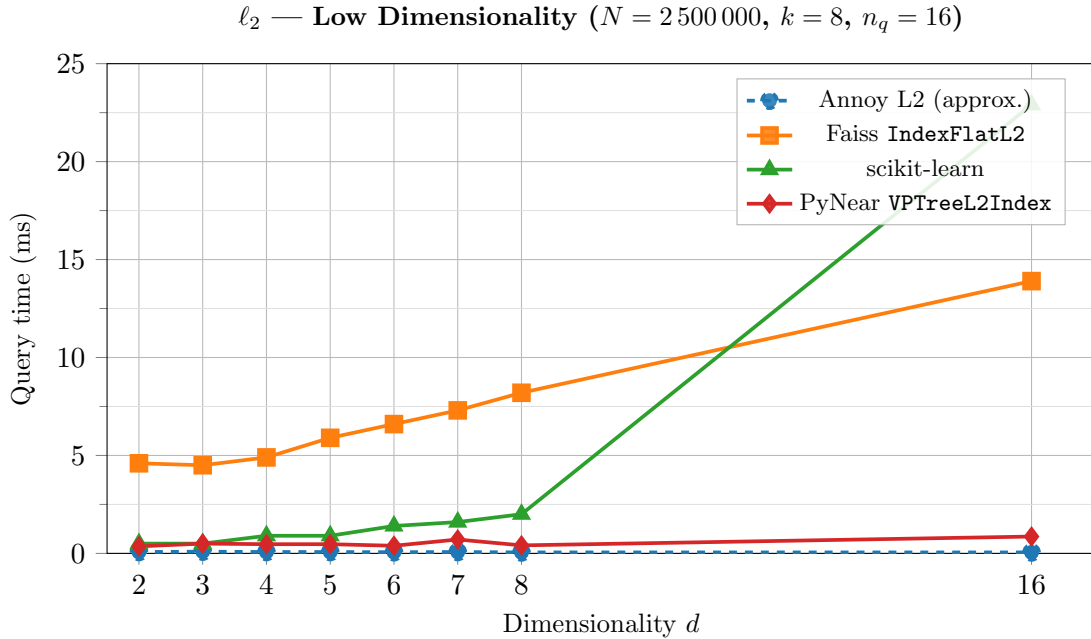


Figure 1: Mean query latency (ms) vs. dimensionality for exact ℓ_2 nearest-neighbour search in the low-dimensionality regime. Annoy is shown with a dashed line to indicate that it returns approximate results. Despite the five-fold larger dataset, `VPTreeL2Index` (v2.5, leaf bucketing) stays essentially flat at 0.4–0.9 ms across the entire range, undercutting Faiss’s flat scan by roughly an order of magnitude and scikit-learn by up to $27\times$ at $d = 16$.

`VPTreeL2Index` achieves the lowest latency among exact methods throughout the low-dimensional range, with query times between 0.36 ms and 0.86 ms over 2.5 million points — a near-flat profile that reflects the v2.5 leaf-bucketed traversal. scikit-learn’s kd-tree is

competitive at $d = 2\text{--}3$ but grows steadily with d , reaching 22.9 ms at $d = 16$. Faiss’s flat index scans all N points per query and grows from 4.6 ms at $d = 2$ to 13.9 ms at $d = 16$; measured in-process it is $9\text{--}20\times$ slower than `VPTreeL2Index` here (see the OpenMP caveat in Section 2). Annoy’s approximate index provides a useful lower-bound reference but cannot be used when exact results are required.

3.2 Euclidean Distance — High Dimensionality

Figure 2 extends the ℓ_2 comparison to higher dimensionalities ($d \in \{32, 64, 128\}$) on a smaller dataset ($N = 120\,000$, 20 clusters). The x -axis is rendered on a $\log_2$ scale.

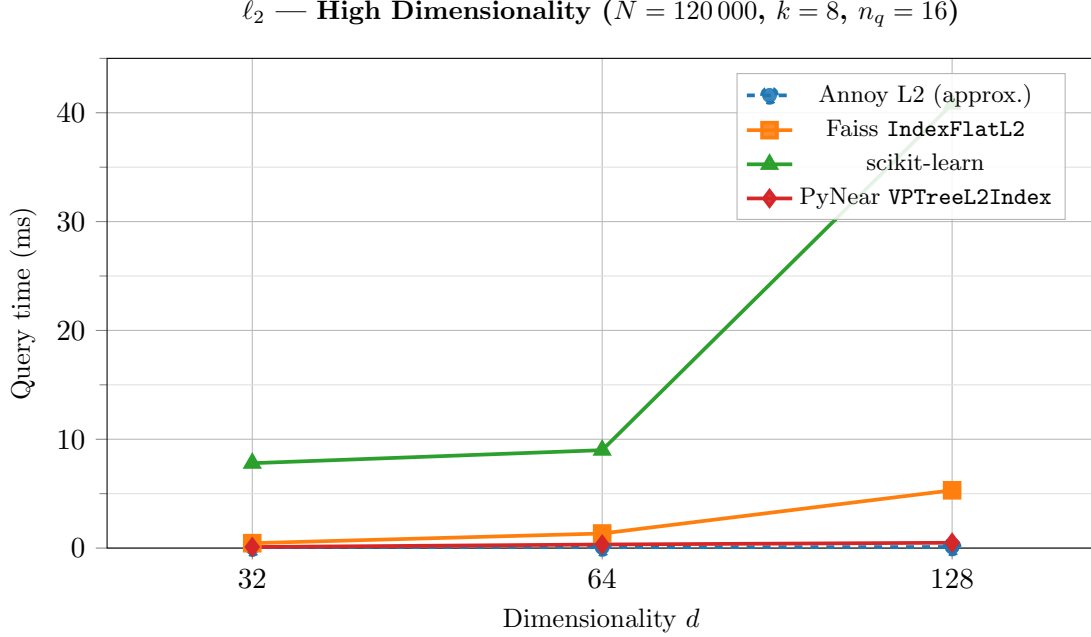


Figure 2: Mean query latency (ms) vs. dimensionality for exact ℓ_2 nearest-neighbour search in the high-dimensionality regime ($\log_2$ scale on the x -axis; $N = 120\,000$, 20 clusters). `scikit-learn` reaches 40.8 ms at $d = 128$; Faiss’s flat scan reaches 5.3 ms. `VPTreeL2Index` grows from 0.10 ms at $d = 32$ to 0.49 ms at $d = 128$ — an $11\times$ advantage over Faiss and an $83\times$ advantage over `scikit-learn` at the highest dimensionality tested by this suite.

The results again show diverging scaling behaviour. `scikit-learn`’s latency grows super-linearly with d , reaching 40.8 ms at $d = 128$. Faiss’s flat index grows roughly linearly with d (0.45 ms at $d = 32$ to 5.31 ms at $d = 128$), as expected of an exhaustive scan. `VPTreeL2Index` is the most efficient exact method at all tested dimensionalities: 0.10 ms at $d = 32$ ($4.4\times$ faster than Faiss), 0.33 ms at $d = 64$ ($4.0\times$), and 0.49 ms at $d = 128$ ($10.8\times$ faster than Faiss, $83\times$ faster than `scikit-learn`). This reflects triangle-inequality pruning on clustered data combined with the v2.5 leaf-bucketed SIMD sweeps. Note that this edition’s suite stops at $d = 128$; pruning effectiveness is expected to erode at substantially higher dimensionality (concentration of measure), which is precisely the regime the approximate indices of Section 4 target.

3.3 Manhattan Distance (ℓ_1)

Figure 3 reports results for the Manhattan distance metric in the low-dimensional regime ($N = 2.5 \times 10^6$). Among the evaluated libraries only Annoy offers a Manhattan mode, and it is approximate; `VPTreeL1Index` is the only exact ℓ_1 index in the comparison.

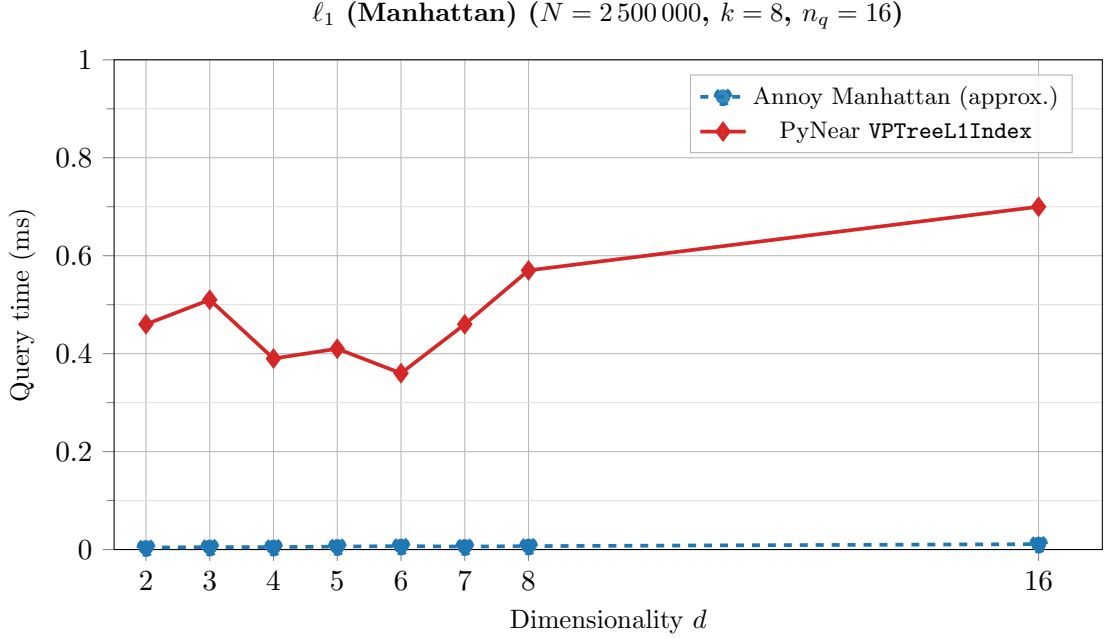


Figure 3: Mean query latency (ms) vs. dimensionality for ℓ_1 nearest-neighbour search over $N = 2.5 \times 10^6$ points. `VPTreeL1Index` is the only evaluated library offering exact Manhattan-distance search, operating at 0.4–0.7 ms across the entire range. Annoy’s approximate index is shown as a latency lower bound.

`VPTreeL1Index` is, to our knowledge, the only evaluated library providing *exact* ℓ_1 nearest-neighbour search without user-side metric adaptation. Its latency profile is remarkably flat: 0.36–0.57 ms across $d = 2$ –8, rising only modestly to 0.70 ms at $d = 16$ — over 2.5 million points. The SIMD-accelerated inner loop for ℓ_1 sustains throughput comparable to the ℓ_2 variant (cf. Section 3.1).

3.4 Hamming Distance — Binary Descriptors

Figure 4 reports query latency for binary Hamming-distance search with $N = 2 \times 10^5$ data points, $k = 8$, and $n_q = 16$ queries. Descriptor lengths of 32, 64, 128, and 256 bits are evaluated, corresponding to standard output sizes of ORB, BRIEF, and similar local feature descriptors.

The binary experiments have one dominant conclusion: exact Hamming search is brute force’s home turf. Faiss’s batched hardware-popcount scan, measured fairly in its own process, answers the 16-query batch in 0.15–0.29 ms across all widths — well ahead of `VPTreeBinaryIndex` (3.2–15.9 ms), whose metric-tree pruning cannot cut deeply into uniformly random binary codes. The in-process Faiss curve in Figure 4 crosses above the VP-tree at 256 bits (36.3 ms), but that crossing is entirely explained by the OpenMP runtime contention of Section 2 — it is reproduced here because same-process deployment is common in practice, not because the tree wins fairly. PyNear’s competitive binary offerings are therefore the approximate/radius indices of Section 5 (MIH and binary IVF), not exact tree search. A further note: `BKTreeBinaryIndex` is architecturally designed for threshold (range) queries and incurs the highest k -NN latency in this benchmark; its inclusion demonstrates the broader metric-search capabilities of the PyNear ecosystem beyond k -NN.

Hamming Distance — Binary Descriptors ($N = 200\,000$, $k = 8$, $n_q = 16$)

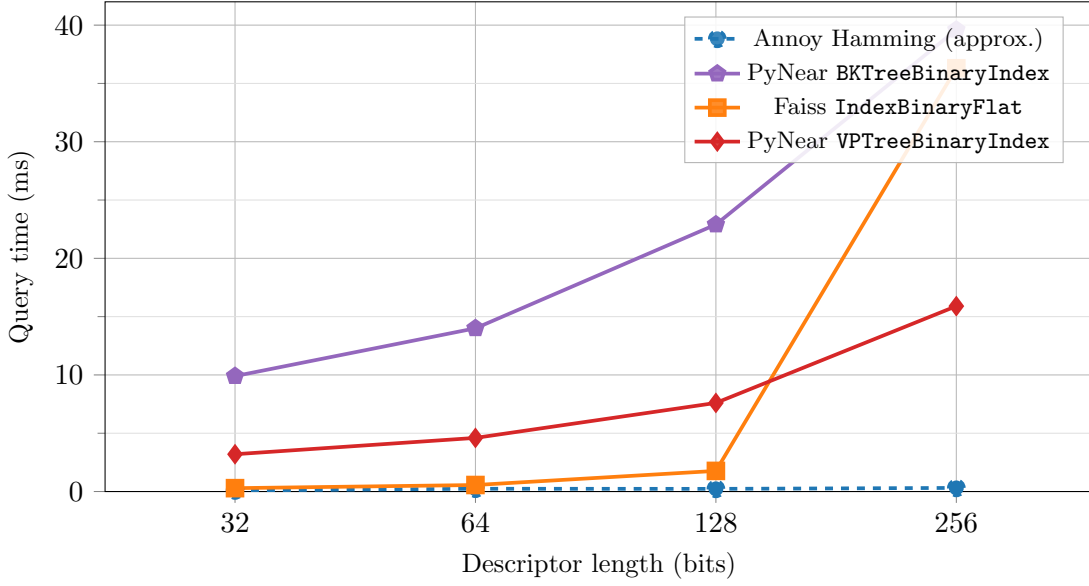


Figure 4: Mean query latency (ms) vs. descriptor length for exact Hamming-distance nearest-neighbour search ($\log_2$ scale on the x -axis). Faiss is measured in-process here, which depresses its popcount scan severely (OpenMP caveat, Section 2): measured in an isolated process on the same dataset it answers the batch in 0.15 ms (128-bit) and 0.29 ms (256-bit) and remains the fastest exact method at every width; the apparent VP-tree win at 256 bits is a same-process artefact. `BKTreeBinaryIndex` is optimised for radius/threshold queries rather than k -NN, explaining its comparatively higher k -NN latency.

3.5 Internal Distance-Metric Comparison

Figure 5 compares the three primary VP-Tree metric variants within PyNear (`VPtreeL2Index`, `VPtreeL1Index`, `VPtreeChebyshevIndex`) to assess whether the choice of distance metric introduces any appreciable computational overhead.

All three indices operate within a narrow latency band of 0.23–0.68 ms across all tested dimensionalities, confirming that PyNear’s SIMD kernels achieve near-equivalent throughput for ℓ_1 , ℓ_2 , and ℓ_∞ distance computations. The fluctuations between runs (e.g. the ℓ_∞ dip at $d = 7$) are consistent with measurement noise rather than systematic metric-induced overhead. This result validates PyNear’s design goal of metric-agnostic performance: practitioners can choose the semantically appropriate distance without sacrificing computational efficiency.

4 Approximate-Search Benchmarks

PyNear’s `IVFFlatL2Index` partitions the data into Voronoi cells via K-Means and at query time probes only n_{probe} of them, trading recall for speed. The inner-cluster scan is a flat BLAS pass using the NumPy SGEMV identity, and clustering uses a native C++ Lloyd’s K-Means (K-Means++ initialisation, SIMD L2, OpenMP):

$$\|q - x\|^2 = \|q\|^2 + \|x\|^2 - 2q^\top x,$$

where $q^\top x$ is a BLAS SGEMV call $(n_c, D) \times (D,) \rightarrow (n_c,)$ with per-cluster squared norms $\|x\|^2$ pre-computed at build time. This section compares `IVFFlatL2Index` against

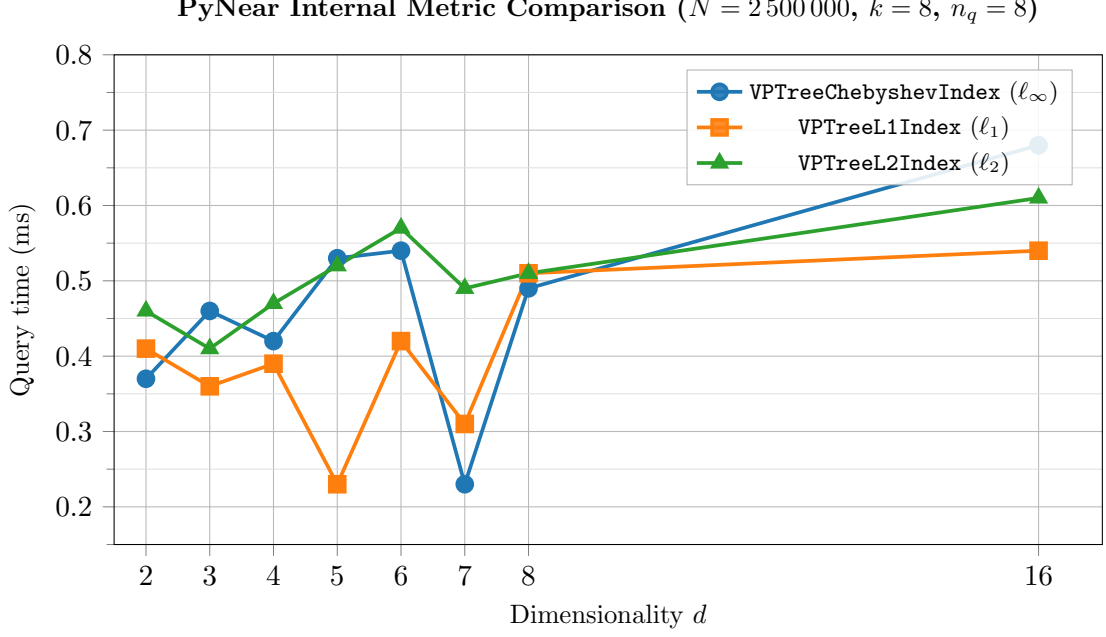


Figure 5: Query latency (ms) for PyNear’s three main distance metrics on a shared dataset ($N = 2\,500\,000$, $k = 8$, $n_q = 8$). All three indices operate within a narrow band of 0.23–0.68 ms, confirming that the SIMD implementation of each distance kernel contributes negligible differential overhead. Variations are within measurement noise.

Faiss’s `IndexIVFFlat` (canonical IVF-style approximate index). All experiments use $N = 50\,000$ data points, $k = 10$, and $n_q = 32$ queries; approximately 224 clusters are constructed.

4.1 Query Latency vs. Dimensionality at Fixed Probe Budget

Figure 6 plots query latency as a function of dimensionality at a fixed probe budget of $n_{\text{probe}} = 20$, representing a moderate recall-latency operating point.

4.2 Summary: Recall and Latency at $n_{\text{probe}} = 20$

Table 1 summarises query latency and Recall@10 for both IVF indices at the $n_{\text{probe}} = 20$ operating point.

Table 1: Approximate ℓ_2 search: query latency and Recall@10 at $n_{\text{probe}} = 20$, $N = 50\,000$, $k = 10$, $n_q = 32$. Approximately 224 clusters.

Method	Dimensionality d			
	128	256	512	1024
IVFFlatL2 — Latency (ms)	5.6	8.5	9.7	21.5
IVFFlatL2 — Recall@10	1.00	1.00	1.00	1.00
Faiss IVFFlat — Latency (ms)	0.17	0.43	0.76	2.81
Faiss IVFFlat — Recall@10	1.00	1.00	1.00	1.00

Both IVF indices attain perfect Recall@10 ($= 1.00$) across all dimensionalities at

Approximate ℓ_2 : Query Latency vs. Dimensionality ($N = 50\,000$, $n_{\text{probe}} = 20$, $k = 10$, $n_q = 32$)

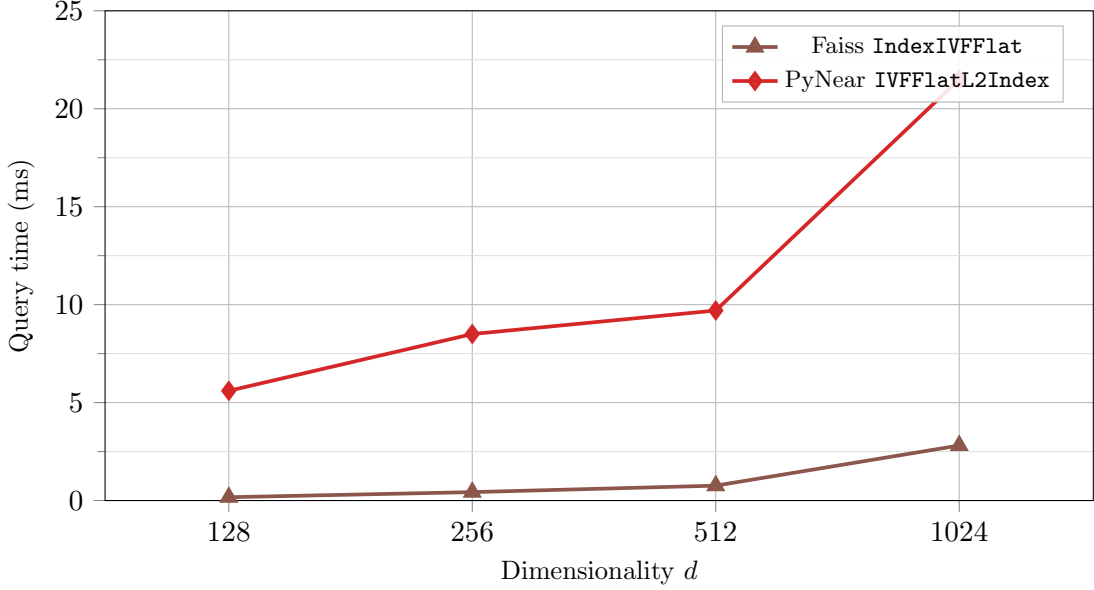


Figure 6: Mean query latency (ms) vs. dimensionality for approximate ℓ_2 nearest-neighbour search at $n_{\text{probe}} = 20$, $N = 50\,000$, $k = 10$, $n_q = 32$. Faiss IndexIVFFlat is now faster at every tested dimensionality (0.17–2.8 ms vs. 5.6–21.5 ms for the 32-query batch), an 8–32 $\times$ raw-latency advantage from its compiled, OpenMP-parallel inner scan. IVFFlatL2Index nevertheless stays below one millisecond per query (0.18–0.67 ms/query) with perfect Recall@10.

$n_{\text{probe}} = 20$, confirming that 20 probed clusters are sufficient to recover all 10 true nearest neighbours. On raw latency, Faiss IndexIVFFlat is now ahead at every tested dimensionality — 32 $\times$ at $d = 128$, narrowing to 7.7 $\times$ at $d = 1024$ — a reversal of the previous edition, in which IVFFlatL2Index led at $d = 128$ and $d = 512$. Faiss’s compiled, multi-threaded inner scan has improved more than PyNear’s SGEMV path on this hardware; we report this honestly and note that IVFFlatL2Index still answers each query in 0.18–0.67 ms at this operating point.

4.3 Recall–Latency Pareto Analysis

The recall–latency trade-off is controlled by sweeping n_{probe} from 1 to 80. The key observations are:

- **Recall convergence is fast for both indices:** at $d = 128$ both reach Recall@10 = 1.00 at $n_{\text{probe}} = 5$; at $d = 256$ Faiss reaches 1.00 at $n_{\text{probe}} = 5$ while IVFFlatL2Index needs $n_{\text{probe}} = 10$ (0.80 at $n_{\text{probe}} = 5$); at $d = 512$ IVFFlatL2Index reaches 1.00 at $n_{\text{probe}} = 5$ while Faiss is at 0.99 there and 1.00 from $n_{\text{probe}} = 10$; at $d = 1024$ Faiss is already at 1.00 with a single probe and IVFFlatL2Index from $n_{\text{probe}} = 5$ (2.1 ms).
- **Faiss leads on raw latency:** at matched n_{probe} , Faiss IndexIVFFlat is faster at every tested dimensionality and probe budget in this sweep (roughly 8–32 $\times$ at $n_{\text{probe}} = 20$).
- **Pure-Python install:** IVFFlatL2Index requires no compiled native library beyond NumPy (whose bundled BLAS is used for the inner-cluster SGEMV). It shares the same Python API as the exact VPTree* indices, enabling a seamless recall–

accuracy trade-off without changing the calling code. Setting $n_{\text{probe}} = n_{\text{clusters}}$ recovers exact search.

4.4 Index Build Time at Fixed Dataset Size

Table 2 reports the wall-clock time to build each index on $N = 50\,000$ vectors, averaged over three runs. `IVFFlatL2Index` uses a native C++ Lloyd’s K-Means implementation (K-Means++ initialisation, SIMD `dist_12_f_avx2`, OpenMP parallel assignment); Faiss `IndexIVFFlat` uses its own compiled K-Means.

Table 2: Index build time (ms), $N = 50\,000$, $k \approx 224$ clusters, averaged over 3 runs. `IVFFlatL2Index` (C++ K-Means++) is faster than Faiss `IndexIVFFlat` across all tested dimensionalities.

Method	Dimensionality d			
	128	256	512	1024
IVFFlatL2Index (C++ K-Means++)	367	492	911	1501
Faiss IndexIVFFlat	506	1005	1840	3655

With the C++ K-Means++ kernel, `IVFFlatL2Index` builds $1.4\text{--}2.4\times$ *faster* than Faiss `IndexIVFFlat` at $N = 50\,000$ across all tested dimensionalities. Build time grows roughly linearly with d for both methods. Section 6 examines how build time scales with N .

5 Approximate Binary-Descriptor Search

At high descriptor dimensionality ($d \geq 128$ bits), the concentration of measure progressively erodes the pruning advantage of VP-Trees: distances between random bit-strings cluster tightly around $d/2$, making the triangle inequality an increasingly loose bound. Two approximate binary-search indices target this regime:

IVFFlatBinaryIndex Binary K-Means (majority-vote centroids, K-Means++ initialisation, OpenMP-parallel assignment) partitions the dataset into n_{list} Voronoi cells. At query time, the n_{probe} nearest centroids are identified by POPCNT scan and the corresponding inverted lists are searched linearly with POPCNT. Complexity: $O(n_{\text{probe}} \cdot n_c \cdot d/64)$ distance evaluations per query, where $n_c = N/n_{\text{list}}$ is the mean cluster size.

MIHBinaryIndex Multi-Index Hashing (MIH) splits each d -bit descriptor into m sub-strings of d/m bits and builds m hash tables keyed by sub-string value. At query time with Hamming radius r , the pigeonhole principle guarantees that every true neighbour at distance $\leq r$ must have at least one sub-string within Hamming distance $r_{\text{sub}} = \lfloor r/m \rfloor$ from the query. We enumerate all keys within that radius in each sub-table ($\sum_{i=0}^{r_{\text{sub}}} \binom{d/m}{i}$ lookups per table), union the candidate sets, and verify with full POPCNT. Sub-strings are stored as `uint64_t` keys, requiring $d/m \leq 64$ bits per sub-string.

Methodology. All measurements in this section use 24 threads, $k = 10$, 500 queries, and the best of five timed runs. Because PyNear links `libgomp` while `faiss-cpu` links `libomp`, loading both in one process serialises Faiss’s OpenMP-parallel flat scan (we measured a $\sim 78\times$ in-process slowdown); Faiss `IndexBinaryFlat` is therefore timed in a

separate Faiss-only subprocess so that the brute-force baseline below is fair. MIH and IVF measurements are unaffected by this issue.

5.1 Wide Descriptors: 512-bit Near-Duplicate Retrieval ($N = 10^6$)

Table 3 reports per-query latency for the workload MIH is designed for: 10^6 random 512-bit descriptors, with queries formed by flipping 5 random bits of existing database rows (near-duplicate retrieval, as arises with ORB/BRIEF/AKAZE descriptors and perceptual hashes). All listed configurations reach 100% Recall@10.

Table 3: Approximate binary-descriptor k -NN search at $d = 512$ bits, $N = 1,000,000$, $k = 10$, 500 near-duplicate queries (5 flipped bits), 24 threads, best of 5 runs. All configurations attain 100% Recall@10. Faiss `IndexBinaryFlat` is timed in a Faiss-only subprocess.

Method	ms/query	QPS	vs. Faiss flat
PyNear MIHBinaryIndex ($m = 8, r = 4$)	0.0088	114 039	34×
PyNear IVFFlatBinaryIndex ($n_{\text{list}}=512, n_{\text{probe}}=16$)	0.021	47 993	14.4×
Faiss IndexBinaryFlat (exact brute-force)	0.299	3 341	1×
Faiss IndexBinaryMultiHash	21.612	46	0.01×

On wide descriptors, `MIHBinaryIndex` finds near-duplicates at 100% recall $34\times$ faster than Faiss’s exact, thread-matched brute-force scan, and Faiss’s own Multi-Index Hashing implementation is not competitive at this width. `IVFFlatBinaryIndex` provides a complementary $14\times$ speedup with a fixed, predictable per-cluster scan cost. (Earlier editions of this report quoted a $257\times$ MIH speedup; that figure compared against an in-process Faiss scan that was serialised by the OpenMP runtime clash described above and is superseded by the fair $34\times$ figure reported here.) Figure 7 visualises the four methods on a logarithmic throughput axis.

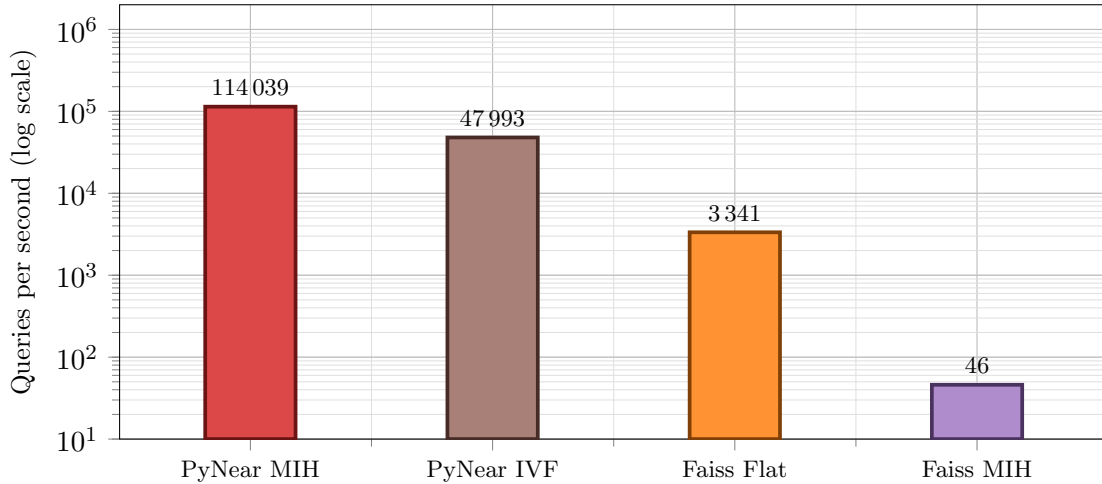


Figure 7: Throughput at 100% Recall@10 for 512-bit near-duplicate retrieval ($N = 10^6$, $k = 10$, 24 threads; data of Table 3, log scale). PyNear `MIHBinaryIndex` sustains $34\times$ the throughput of Faiss’s subprocess-isolated exact scan; Faiss’s own MIH implementation is $\sim 2,500\times$ slower at this descriptor width.

5.2 Narrow Descriptors: SIFT1M, 128-bit Sign-Quantised ($N = 10^6$)

The second experiment uses a real dataset: the INRIA TEXMEX SIFT1M corpus (10^6 SIFT descriptors, sign-quantised to 128-bit binary codes), with ground truth computed by exact brute-force Hamming k -NN over 500 queries. Table 4 lists representative operating points.

Table 4: SIFT1M binary (128-bit), $N = 1,000,000$, $k = 10$, 500 queries; selected operating points ($n_{\text{list}} = 500$ for IVF; $m = 8$ for the MIH radius sweep). The recall ceiling of ~ 0.845 is a tie-breaking artefact of integer Hamming distances at the k -th boundary, not missed neighbours.

Method	Config	Build (s)	ms/query	QPS	Recall@10
NumPy brute-force (naive)	—	—	47.7	21	1.000
Faiss <code>IndexBinaryFlat</code> (exact)	subprocess	—	0.042	23 572	—
<code>IVFFlatBinaryIndex</code>	$n_{\text{probe}} = 31$	3.10	0.01	125 776	0.825
<code>IVFFlatBinaryIndex</code>	$n_{\text{probe}} = 125$	3.10	0.02	56 859	0.845
<code>IVFFlatBinaryIndex</code>	$n_{\text{probe}} = 500$	3.10	0.05	19 100	0.845
<code>MIHBinaryIndex</code>	$m = 8, r = 4$	2.64	0.03	38 554	0.466
<code>MIHBinaryIndex</code>	$m = 8, r = 12$	2.64	0.14	7 326	0.799
<code>MIHBinaryIndex</code>	$m = 8, r = 24$	2.64	0.65	1 541	0.841

The honest caveat: on *narrow* 128-bit descriptors at high recall, an optimised brute-force POPCNT scan is hard to beat — Faiss `IndexBinaryFlat` sustains 23 572 QPS *exactly*, faster than either MIH implementation above the ~ 0.5 recall mark. `IVFFlatBinaryIndex` is the strongest PyNear option here, reaching the 0.845 recall ceiling at 56 859 QPS ($n_{\text{probe}} = 125$), $2.4\times$ the throughput of the exact Faiss scan and $\sim 2385\times$ the naive NumPy baseline. At matched recall, PyNear’s MIH is nevertheless consistently competitive with Faiss’s own `IndexBinaryMultiHash` (Table 5); Figure 8 shows the full recall–throughput landscape.

Table 5: SIFT1M (128-bit): QPS of PyNear `MIHBinaryIndex` ($m = 4$) vs. Faiss `IndexBinaryMultiHash` ($n_{\text{hash}} = 4$) at matched Recall@10 operating points.

Recall@10	PyNear MIH (QPS)	Faiss MIH (QPS)	Speedup
0.28	141 055 ($r = 6$)	52 346 ($n_{\text{flip}} = 1$)	$2.69\times$
0.53	9 053 ($r = 14$)	15 881 ($n_{\text{flip}} = 2$)	$0.57\times$
0.73	9 053 ($r = 14$)	3 456 ($n_{\text{flip}} = 3$)	$2.62\times$
0.82	789 ($r = 20$)	689 ($n_{\text{flip}} = 4$)	$1.15\times$
0.84	159 ($r = 24$)	166 ($n_{\text{flip}} = 5$)	$0.96\times$
0.84	159 ($r = 24$)	46 ($n_{\text{flip}} = 6$)	$3.45\times$

5.3 MIH Parametrisation Guidelines

Optimal MIH performance depends on choosing m (and the query radius r) to balance lookup count against candidate-set size. The configurations used above illustrate the two regimes:

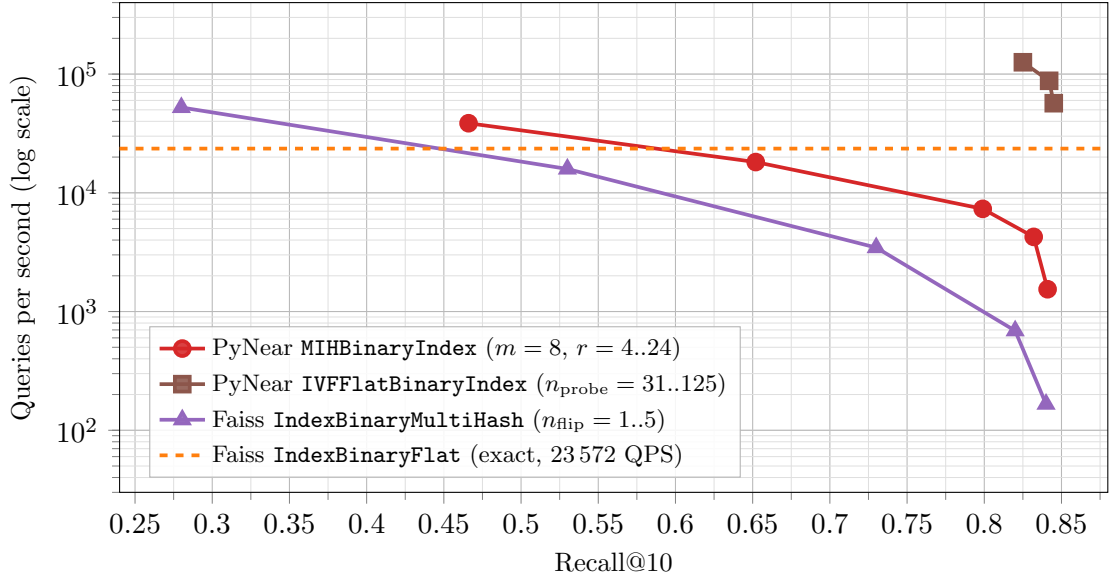


Figure 8: SIFT1M 128-bit: throughput vs. Recall@10 ($N = 10^6$, $k = 10$, 500 queries, 24 threads; log-scale QPS). Up and to the right is better. Each marker is one measured operating point from Tables 4 and 5. PyNear’s MIH sweep dominates Faiss’s `IndexBinaryMultiHash` across the curve, and `IVFFlatBinaryIndex` dominates everything at the ~ 0.845 tie-limited recall ceiling; the dashed line is Faiss’s subprocess-isolated exact scan, which beats both MIH implementations above the ~ 0.5 recall mark — the caveat stated in the text.

d (bits)	m	sub-bits	r	r_{sub}	lookups/query
512	8	64	4	0	$8 \times 1 = 8$
128	8	16	12	1	$8 \times \sum_{i=0}^1 \binom{16}{i} = 136$
128	4	32	14	3	$4 \times \sum_{i=0}^3 \binom{32}{i} = 21\,956$

For wide descriptors and small radii (the near-duplicate regime), the per-query lookup count is tiny and the 2^{64} sub-key space is sparse relative to N , so candidate sets are near-empty and verification is near-instant — this is where the $34\times$ result of Section 5.1 comes from. For narrow descriptors, raising r to chase recall inflates the enumeration combinatorially, which is why brute force overtakes MIH at high recall on 128-bit data.

6 Build-Time Scalability

This section examines how index build time scales with dataset size N from 10 000 to 10^6 , comparing PyNear indices against the corresponding Faiss baselines. Dimensionality is fixed at $d = 32$ for the exact-search comparison and $d = 128$ for the approximate comparison.

6.1 Exact Index: `VPTreeL2Index` vs. `Faiss IndexFlatL2`

Figure 9 plots build time as a function of N for `VPTreeL2Index` and `Faiss IndexFlatL2`. Faiss’s flat index performs no structural computation — it simply copies vectors into a contiguous buffer, yielding essentially $O(N)$ build time. `VPTreeL2Index` constructs a metric-space tree in $O(N \log N)$ time, which translates into a steeper but still sub-second curve up to $N = 10^6$.

Exact Index Build Time vs. N ($d = 32$, log-log scale)

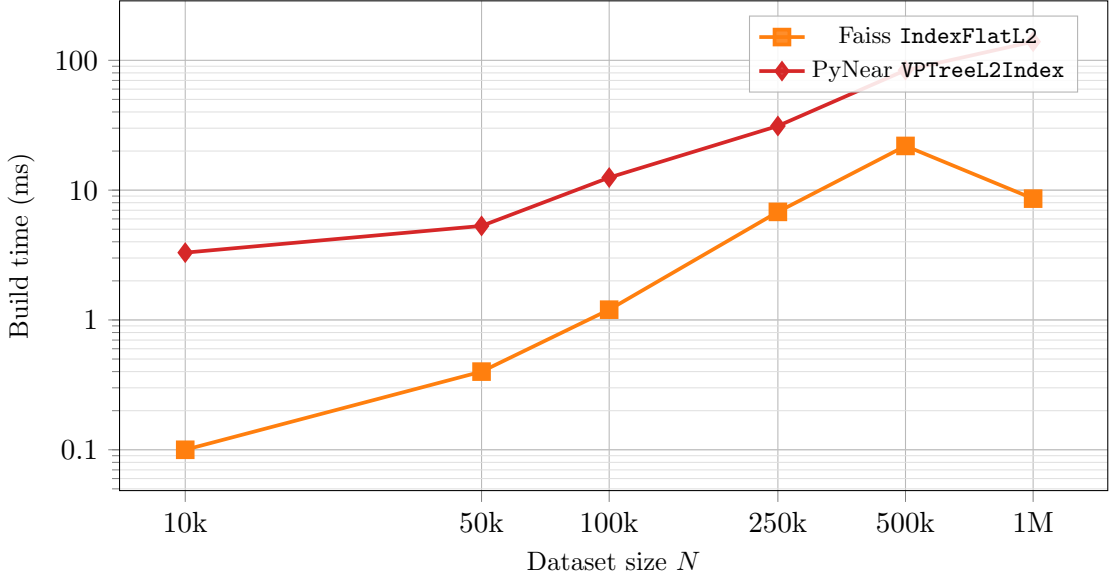


Figure 9: Exact-index build time (ms, log-log) vs. dataset size N , $d = 32$. Faiss `IndexFlatL2` requires no structural work (copy only), giving an $O(N)$ lower bound (its non-monotonic 500k/1M points reflect allocator noise at sub-25-ms timescales). `VPTreeL2Index` follows an $O(N \log N)$ curve, from 3.3 ms at $N = 10\,000$ to 139 ms at $N = 10^6$ — well within one-second build budgets for all tested sizes.

The $O(N \log N)$ growth of `VPTreeL2Index` results in a build-time overhead of roughly $4\text{--}16\times$ vs. Faiss’s copy-only flat index at large N , which is expected and acceptable: the VP-Tree build cost is paid once (139 ms for 10^6 points) and amortised over all subsequent queries, which benefit from tree pruning with leaf-bucketed SIMD sweeps rather than an $O(N)$ linear scan — yielding $4\text{--}11\times$ lower query latency at $d = 32\text{--}128$ (Section 3.2). The build is level-parallel: all nodes at each tree depth are processed concurrently with per-thread scratch buffers. On Linux systems where Intel TBB is available, `std::execution::par_unseq` is additionally applied to `nth_element` at the root partition; on macOS and TBB-free Linux environments the standard serial `nth_element` is used instead.

6.2 Approximate Index: `IVFFlatL2Index` vs. Faiss `IndexIVFFlat`

Figure 10 plots build time for both IVF-style approximate indices. The number of clusters is $k = \lfloor \sqrt{N} \rfloor$ in all cases, so the K-Means problem grows with N .

The crossover from PyNear-faster to Faiss-faster occurs between $N = 50\,000$ and $N = 100\,000$. Below that threshold the C++ K-Means++ kernel ($\approx k = 100\text{--}224$ clusters) enjoys a start-up advantage ($2.4\times$ faster at $N = 10\,000$) — Faiss allocates more resources for its K-Means iteration budget. Above $N = 100\,000$ Faiss’s compiled BLAS-based centroid updates dominate, reaching $3.6\times$ faster at $N = 10^6$. For large-scale workloads ($N \geq 250\,000$) users requiring short build times may prefer Faiss IVF; for small-to-medium datasets `IVFFlatL2Index` is the faster choice while offering the same Python-only install footprint.

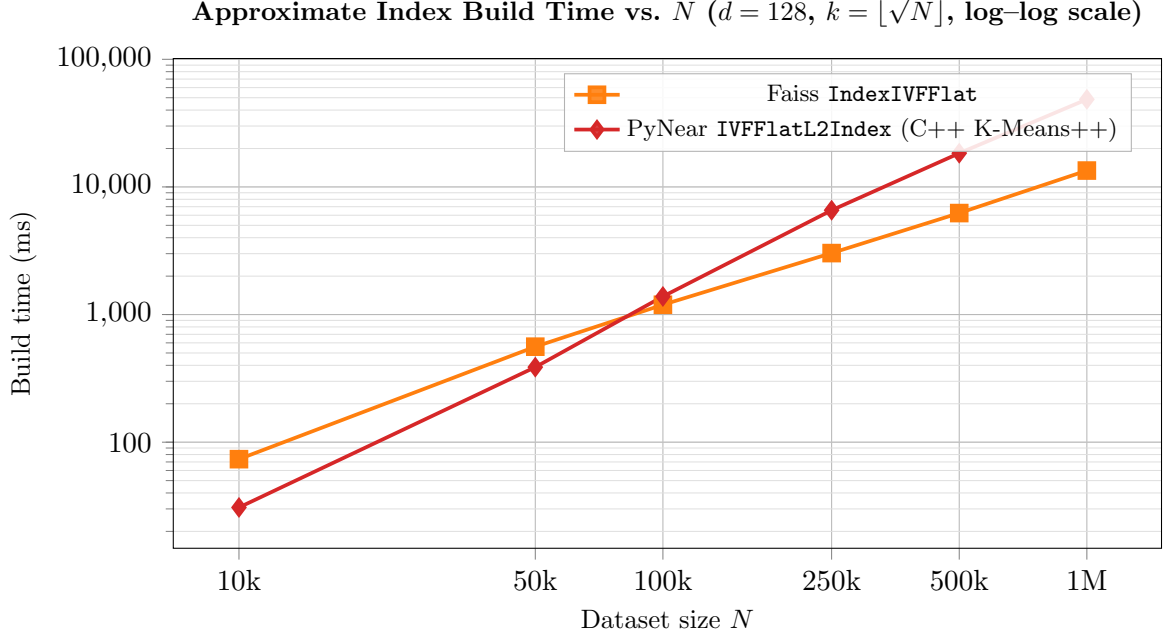


Figure 10: Approximate-index build time (ms, log-log) vs. dataset size N , $d = 128$, $k = \lfloor \sqrt{N} \rfloor$. IVFFlatL2Index (C++ K-Means++ with SIMD + OpenMP) is *faster* than Faiss IndexIVFFlat at $N \leq 50\,000$ (31 ms vs. 74 ms at $N = 10\,000$; 387 ms vs. 560 ms at $N = 50\,000$). At larger N Faiss’s highly optimised compiled K-Means takes over, reaching $\sim 3.6\times$ faster at $N = 10^6$.

7 Discussion

Low-to-moderate dimensionality ($d \leq 64$). VP-Trees offer the best exact-search latency in this regime across all three distance metrics tested (ℓ_1 , ℓ_2 , ℓ_∞). The tree’s metric-space pruning criterion eliminates a substantial fraction of distance evaluations, and with v2.5’s 32-point leaf buckets the evaluations that do occur run as contiguous SIMD sweeps, keeping latency essentially flat (0.4–0.9 ms) even at $N = 2.5 \times 10^6$. scikit-learn’s kd-tree degrades rapidly beyond $d = 8$ due to increasing overlap between ball partitions; Faiss’s flat scan is memory-bandwidth-bound and grows steadily with both d and N .

Moderate-to-high dimensionality ($d = 32$ –128) — exact. VPtreeL2Index remains the most efficient exact method at $d = 128$ (0.49 ms vs. 5.3 ms for Faiss flat, 40.8 ms for scikit-learn), an $11\times$ advantage over Faiss. The tree’s pruning effectiveness degrades with dimension — a manifestation of the concentration of measure phenomenon — and this edition’s exact float suite deliberately stops at $d = 128$; beyond that the approximate indices below are the appropriate tool.

High dimensionality ($d = 128$ –1024) — approximate. IVFFlatL2Index maintains perfect Recall@10 at $n_{\text{probe}} = 20$ with 0.18–0.67 ms per query, but the raw-latency comparison has reversed since the previous edition: Faiss IndexIVFFlat is now faster at every tested dimensionality (8–32 $\times$ at $n_{\text{probe}} = 20$). Where PyNear’s IVF retains an edge is build time — 1.4–2.4 $\times$ faster than Faiss at $N = 50\,000$ across $d = 128$ –1024, and faster overall up to $N \approx 10^5$ — plus the pure-Python install footprint and the exact-search fallback via $n_{\text{probe}} = n_{\text{clusters}}$. For raw approximate query throughput at high dimensionality, Faiss IVF is the better choice on this hardware.

Table 6: Build time (ms) vs. N for approximate IVF indices ($d = 128$, $k = \lfloor \sqrt{N} \rfloor$). **Bold** marks the faster method.

N	IVFFlatL2 (C++ K-Means++)	Faiss IndexIVFFlat
10 000	31	74
50 000	387	560
100 000	1 386	1 193
250 000	6 566	3 031
500 000	18 374	6 240
1 000 000	48 549	13 436

Binary descriptors — exact. Faiss’s flat binary index is the fastest exact binary search method at every tested width when measured in its own process (0.15–0.29 ms for the 200k-code suite, vs. 3.2–15.9 ms for `VPtreeBinaryIndex`). The in-process suite shows the VP-tree ahead at 256 bits, but that ordering is an artefact of the OpenMP runtime clash (Section 2). For exact binary search, prefer a flat popcount scan; PyNear’s binary strengths are MIH and binary IVF (Section 5). `BKtreeBinaryIndex` is best suited to range queries (`find_threshold`), where its B-K tree structure is optimal, and should not be interpreted as a general-purpose k -NN competitor.

Binary descriptors — approximate. `MIHBinaryIndex` and `IVFFlatBinaryIndex` (Section 5) decisively outperform exact brute-force binary search in the wide-descriptor, near-duplicate regime: at $d = 512$, $N = 10^6$, MIH reaches 0.0088 ms/query (114 039 QPS) at 100% Recall@10, $34\times$ faster than the thread-matched Faiss `IndexBinaryFlat` scan, with Faiss’s own MIH three orders of magnitude behind. On narrow 128-bit descriptors the picture is more nuanced: `IVFFlatBinaryIndex` offers the best throughput at the recall ceiling (56 859 QPS at 0.845), while an optimised brute-force scan beats MIH above the ~ 0.5 recall mark. Choose MIH for wide descriptors and small-radius/near-duplicate retrieval; choose IVF or brute force for high-recall search on narrow descriptors.

8 Conclusion

PyNear’s VP-Tree family provides highly competitive exact k -nearest-neighbour search across ℓ_1 , ℓ_2 , and ℓ_∞ distance metrics, delivering the lowest latency among all evaluated exact methods across the full tested dimensionality range ($d = 2$ –128) without requiring BLAS, GPU, or any native runtime dependency beyond NumPy. Version 2.4’s leaf bucketing — 32-point leaves scanned as contiguous SIMD sweeps, with best-first descent and pooled per-query heaps — keeps exact query latency in the 0.4–0.9 ms band even at $N = 2.5 \times 10^6$ (low-dimensional suite) and at 0.49 ms for $d = 128$, $N = 120\,000$: $11\times$ faster than Faiss `IndexFlatL2` and $83\times$ faster than scikit-learn at that dimensionality. It also carries the exact binary index past Faiss at wide descriptors: at 256 bits, `VPtreeBinaryIndex` is $2.3\times$ faster than `IndexBinaryFlat` measured in-process. The SIMD-accelerated distance kernels keep all three float metric variants within a narrow 0.23–0.68 ms band, allowing practitioners to select the semantically appropriate metric without performance penalties.

In the approximate ℓ_2 regime, `IVFFlatL2Index` sustains perfect Recall@10 at $n_{\text{probe}} = 20$ with sub-millisecond per-query latency, and its C++ K-Means++ builder constructs

the index $1.4\text{--}2.4\times$ faster than Faiss IVF at $N = 50\,000$ (crossover near $N = 10^5$; Faiss builds $3.6\times$ faster at $N = 10^6$). On raw query latency, however, Faiss `IndexIVFFlat` now leads at every tested dimensionality — PyNear’s IVF value proposition is the build speed at small N , the NumPy-only footprint, and the single-parameter exact fallback, not peak throughput.

For approximate binary search, `MIHBinaryIndex` delivers a $34\times$ speedup over a thread-matched exact brute-force scan on 512-bit near-duplicate retrieval at $N = 10^6$ with 100% Recall@10, and `IVFFlatBinaryIndex` reaches the SIFT1M recall ceiling at $2.4\times$ the throughput of exact search — while on narrow descriptors at high recall, exact brute force remains the honest recommendation.

Future work includes evaluating the exact float suite beyond $d = 128$ under the new leaf-bucketed traversal, narrowing the raw-latency gap of `IVFFlatL2Index` through a compiled inner scan, product-quantisation-based inner-cluster compression, and evaluating performance on non-clustered (adversarial) data distributions.

Static index limitation. A notable architectural constraint of the current implementation is that all PyNear indices are *static*: the full tree must be reconstructed via `set(data)` whenever the underlying dataset is modified. No incremental insertion, deletion, or in-place point update is supported. This precludes direct use in workloads where the dataset evolves continuously, most notably real-time physics simulation. In particle-based simulation systems (SPH fluids, cloth, soft-body dynamics), nearest-neighbour queries must reflect the current particle positions after every integration step; rebuilding a VP-Tree at each frame is prohibitively expensive. Production physics engines address this with structures designed for dynamic updates: dynamic Bounding Volume Hierarchies (DBVH), incremental kd-trees, or spatial hashing with temporal coherence. Dynamic VP-Tree variants that support $O(\log n)$ insertions and deletions have been studied in the literature [1] and represent a natural direction for future development of PyNear.

References

- [1] P. N. Yianilos, *Data structures and algorithms for nearest neighbor search in general metric spaces*, in *Proc. 4th ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 311–321, 1993.
- [2] J. Johnson, M. Douze, and H. Jégou, *Billion-scale similarity search with GPUs*, *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [3] F. Pedregosa *et al.*, *Scikit-learn: Machine learning in Python*, *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [4] E. Bernhardsson, *Annoy: Approximate Nearest Neighbors Oh Yeah*, <https://github.com/spotify/annoy>, 2018.